

PSEUDOWEB

Manual del alumno

Compatible con la sintaxis de PSeInt · 36 ejercicios corregidos automáticamente

Versión	PseudoWeb v0.6
Fecha	14 / 05 / 2026
Idioma	Español
Soporta	Linux · macOS · Windows (vía PSeInt) + cualquier navegador moderno (PseudoWeb)
Licencia	Uso educativo

Una herramienta para introducir el pensamiento algorítmico en español

Índice de contenidos

PARTE I	Sobre PSeInt y PseudoWeb
§1.1	¿Qué es PSeInt? Descarga oficial para Windows / macOS / Linux
§1.2	¿Qué es PseudoWeb? Diferencias y ventajas
§1.3	Atajos y flujo de trabajo típico
PARTE II	Referencia de sintaxis del pseudocódigo
§2.1	Estructura del programa (Proceso/FinProceso)
§2.2	Variables y tipos (Definir ... Como ...)
§2.3	Entrada / Salida (Escribir, Leer)
§2.4	Operadores aritméticos, relacionales y lógicos
§2.5	Estructuras de control (Si, Segun, Mientras, Repetir, Para)
§2.6	Arreglos (Dimension)
§2.7	Funciones built-in (matemáticas, aleatorias, cadenas)
PARTE III	Catálogo de 36 ejercicios
§3.1	Cómo se evalúa: 10 casos por ejercicio, nota sobre 10
§3.2	① Básico
§3.3	② Condicional simple
§3.4	③ Condicional múltiple
§3.5	④ Bucle Para
§3.6	⑤ Bucle Mientras
§3.7	⑥ Algoritmos clásicos
§3.8	⑦ Listas (Dimension)
§3.9	⑧ Menús con Segun
§3.10	⑨ Bucle Repetir / Hasta Que
§3.11	⑩ Números aleatorios
§3.12	■ Cadenas de texto
§3.13	■ Funciones matemáticas
PARTE IV	Apéndices
§4.1	Errores comunes y cómo solucionarlos
§4.2	Tabla resumen de constructos

PARTE I — Sobre PSeInt y PseudoWeb

1.1 ¿Qué es PSeInt? Descarga oficial

PSeInt (Pseudo-código Intérprete) es un programa de escritorio creado por Pablo Novara en la Universidad Nacional del Litoral (Argentina, ~2003). Permite escribir, ejecutar y depurar algoritmos en pseudocódigo en español. Es la herramienta de referencia en cursos universitarios de introducción a la programación en toda Latinoamérica y España.

Software libre (GPL), multiplataforma. Descarga oficial:

Sistema operativo	Enlace de descarga	Notas
Windows 7 / 10 / 11	sourceforge.net/projects/pseint/files/ · busca "Windows"	Instalador .exe (~30 MB) o ZIP portable
macOS 10.13+	sourceforge.net/projects/pseint/files/ · busca "MacOS"	Archivo .dmg para arrastrar a Aplicaciones
Linux x86_64	sourceforge.net/projects/pseint/files/ · busca "Linux"	Applimage portable o .tar.gz. En Fedora: <code>sudo dnf install pseint</code> (en algunos repos)
Página oficial	https://pseint.sourceforge.io/	Documentación, capturas, foro de ayuda

Verifica siempre la integridad del fichero comparando el SHA-256 antes de instalar; la página de SourceForge publica los hashes oficiales. Evita descargar PSeInt desde sitios que no sean SourceForge o pseint.sourceforge.io.

1.2 ¿Qué es PseudoWeb?

PseudoWeb es un IDE web complementario a PSeInt: se abre con doble clic sobre `index.html` en cualquier navegador moderno (Firefox, Chrome, Edge, Safari) y no requiere instalación. Está pensado como apoyo en clase, en aulas sin permisos de instalación, o cuando se trabaja desde un ordenador prestado.

Diferencias resumidas:

Aspecto	PSeInt (escritorio)	PseudoWeb (navegador)
Instalación	Sí (Win/macOS/Linux)	No: doble clic en HTML
Sintaxis	Pseudocódigo PSeInt	Compatible PSeInt (con limitaciones)
Diagrama de flujo	Sí, automático	No (pendiente)
Depurador paso a paso	Sí	No (pendiente)
Funciones definidas por el usuario	Sí	No (solo built-in)
Exportar a C++ y otros lenguajes	Sí	No
Sistema de ejercicios corregidos	No nativo	Sí: 36 ejercicios, nota 0-10
Autoguardado	Manual	Automático en localStorage

Recomendación: usa PSeInt en casa o en aulas con instalación permitida (es más completo); usa PseudoWeb para practicar ejercicios con corrección automática y para escenarios donde no puedas instalar nada. La sintaxis es compatible: el código que escribes en uno se ejecuta casi siempre tal cual en el otro.

1.3 Atajos y flujo de trabajo en PseudoWeb

Atajo / botón	Qué hace
Ctrl + Enter ó F5	Ejecuta el programa actual
Tab	Inserta 4 espacios (indentación)
Botón "+ Nuevo"	Borra el editor y carga la plantilla Proceso/FinProceso
Selector "Cargar ejemplo"	38 ejemplos en 12 grupos didácticos
Botón "■ Ejercicios"	Catálogo de 36 ejercicios con corrección automática
Botón "Limpiar salida"	Vacía el panel derecho sin afectar al editor
Escape (en modales)	Cierra el modal abierto

El editor guarda su contenido automáticamente cada 400 ms en el almacenamiento del navegador (localStorage). Si recargas la página o vuelves al día siguiente, tu último programa sigue ahí.

PARTE II — Referencia de sintaxis

2.1 Estructura del programa

Todo programa empieza con `Proceso NombreDelPrograma` y termina con `FinProceso`. `Algoritmo/FinAlgoritmo` es un alias aceptado. El nombre del proceso no afecta a la ejecución, pero es buena práctica que describa la tarea.

```
Proceso HolaMundo
Escribir "¡Hola, mundo!"
FinProceso
```

2.2 Variables y tipos

Las variables se declaran con `Definir nombre Como Tipo`. Se pueden declarar varias a la vez separadas por coma.

Tipo	Valores que admite	Ejemplo
Entero	Números enteros con signo. Truncan al asignar.	Definir n Como Entero
Real	Números con decimales (coma flotante)	Definir x Como Real
Numero	Alias genérico para Real	Definir y Como Numero
Caracter / Cadena / Texto	Cadenas de caracteres entre comillas dobles	Definir nombre Como Cadena
Logico	Verdadero o Falso	Definir ok Como Logico

La asignación se hace con `<-` (no `=`). Las variables declaradas como Entero truncan automáticamente al asignar: `n <- 7 / 2` deja `n = 3`.

2.3 Entrada / Salida

`Escribir` imprime uno o más valores (separados por comas). `Leer` espera a que el usuario teclee un valor.

```
Definir nombre Como Cadena
Definir edad Como Entero
Escribir "¿Cómo te llamas?"
Leer nombre
Escribir "¿Cuántos años tienes?"
Leer edad
Escribir "Hola, ", nombre, ". Tienes ", edad, " años."
```

2.4 Operadores

Categoría	Operadores	Ejemplo
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>^</code> <code>MOD</code>	<code>a MOD b</code> $\rightarrow$ resto de a entre b
Relacionales	<code>=</code> <code><></code> <code><</code> <code>></code> <code><=</code> <code>>=</code>	Si <code>a >= 5</code> Entonces ...
Lógicos	<code>Y</code> <code>O</code> <code>NO</code>	Si <code>a > 0 Y b > 0</code> Entonces ...

Asignación	<-	x <- 5 + 3
------------	----	------------

*Precedencia (de mayor a menor): unario "–" · ^ · * · /, MOD · +, – · relacionales · NO · Y · O. Los paréntesis se evalúan primero — úsalos si dudas.*

2.5 Estructuras de control

Si / SiNo / FinSi

```
Si edad >= 18 Entonces
  Escribir "Mayor de edad"
SiNo
  Escribir "Menor de edad"
FinSi
```

Segun / FinSegun — selector múltiple

```
Segun opcion Hacer
1: Escribir "Uno"
2, 3: Escribir "Dos o tres"
De Otro Modo: Escribir "Cualquier otro"
FinSegun
```

Mientras / FinMientras — comprueba al principio

```
i <- 1
Mientras i <= 10 Hacer
  Escribir i
  i <- i + 1
FinMientras
```

Repetir / Hasta Que — comprueba al final (se ejecuta ≥ 1 vez)

```
Repetir
  Escribir "Introduce 0 para salir:"
  Leer n
Hasta Que n = 0
```

Para — iteración con contador

```
Para i <- 1 Hasta 10 Hacer
  Escribir i
FinPara

// con paso negativo:
Para i <- 10 Hasta 1 Con Paso -1 Hacer
  Escribir i
FinPara
```

2.6 Arreglos (Dimension)

Dimension v[N] reserva memoria para un vector de N elementos. Los índices van de 1 a N (no de 0 como en muchos lenguajes).

```

Dimension v[10]
Para i <- 1 Hasta 10 Hacer
Leer v[i]
FinPara
// luego usas v[1], v[2], ..., v[10]

```

2.7 Funciones built-in

Categoría	Funciones	Notas
Matemáticas	RC, Trunc, Redon, Abs, Ln, Exp, Sen, Cos, Tan	Ángulos en radianes
Aleatorias	Azar(N) $\rightarrow$ 0..N-1 Aleatorio(a, b) $\rightarrow$ a..b inclusive	Sin semilla controlable
Cadenas	Longitud, Mayusculas, Minusculas, Subcadena(s, i, j), Concatenar, ConvertirANumero, ConvertirATexto	Subcadena es 1-indexada e inclusiva
Constante	PI	Se usa sin paréntesis

PARTE III — Catálogo de 36 ejercicios

3.1 Cómo se evalúa cada ejercicio

Cada ejercicio tiene 10 casos de prueba (combinaciones de entradas con su salida esperada). El grader ejecuta tu programa una vez por cada caso, alimentándole las entradas, y compara la última línea (o las últimas N líneas) de salida con la esperada.

Aciertos	Nota	Indicador
10 de 10	10 / 10 — perfecto	■
5 a 9	5..9 / 10 — casi	■
1 a 4	1..4 / 10 — lejos	■
Sin intentar	— / 10	■

La comparación es estricta: "Suma: 10" no coincide con "suma: 10" (mayúscula) ni con "Suma:10" (sin espacio). Lee con cuidado el enunciado y respeta los espacios y mayúsculas exactos. Tu programa *Sí* puede imprimir prompts antes (p. ej. "Introduce N:") — solo se evalúa la última línea.

Los ejercicios del grupo ⑩ (Aleatorios) NO comprueban un valor exacto (sería imposible). Verifican el formato de la salida y que el resultado esté en el rango matemáticamente válido.

3.2 ① Básico

Programas con entrada/salida y una operación aritmética. Ideal para familiarizarse con Definir, Leer y Escribir.

Perímetro del cuadrado (id: perimetro_cuadrado · 10 casos)

Pide al usuario la longitud del lado de un cuadrado y calcula su perímetro ($4 \times \text{lado}$).

Como ÚLTIMA línea debes imprimir exactamente:

Perímetro: X

donde X es el valor calculado. Puedes (y debes) imprimir prompts antes — solo se evalúa la última línea.

Pistas:

- El perímetro de un cuadrado es $4 \times \text{lado}$
- Usa Definir lado, p Como Real
- Imprime así: Escribir "Perímetro: ", p

Doble y triple (id: doble_triple · 10 casos)

Pide un número y muestra su doble y su triple en una sola línea.

Como ÚLTIMA línea imprime exactamente:

Doble: X, Triple: Y

Pistas:

- El doble es $2 * n$
- El triple es $3 * n$
- Imprime los dos valores en la misma Escribir, separados por coma

Minutos a segundos (id: min_a_seg · 10 casos)

Pide un número de minutos y muestra cuántos segundos son (1 minuto = 60 segundos).

Como ÚLTIMA línea imprime exactamente:
Segundos: X

Pistas:

- Multiplica por 60
- No olvides el formato "Segundos: "

3.3 ② Condicional simple

Un único Si/SiNo. Aprende a leer una condición y decidir entre dos caminos.

¿Es múltiplo de 3? (id: multiplo_3 · 10 casos)

Pide un número entero y muestra como ÚLTIMA línea:

SI — si es múltiplo de 3

NO — si no lo es

(Recuerda: 0 sí es múltiplo de 3 porque $0 = 3 \times 0$.)

Pistas:

- Usa el operador MOD: $n \text{ MOD } 3 = 0$ indica múltiplo de 3
- Solo necesitas un Si/SiNo
- Imprime "SI" o "NO" (mayúsculas, sin nada más)

¿Aprobado? (id: aprobado · 10 casos)

Pide una nota (0..10) y muestra "APROBADO" si la nota es ≥ 5 , o "SUSPENSO" si es menor.

Como ÚLTIMA línea imprime exactamente:
APROBADO o SUSPENSO

Pistas:

- Un Si simple con la condición $\text{nota} \geq 5$
- "APROBADO" y "SUSPENSO" en mayúsculas, sin nada más

Mayor de dos números (id: mayor_dos · 10 casos)

Pide dos números. Muestra el mayor de ambos. Si son iguales, muestra "Son iguales".

Como ÚLTIMA línea imprime exactamente:

Mayor: X (si uno es mayor)

Son iguales (si $a = b$)

Pistas:

- Si $a > b$ Entonces ... SiNo Si $a < b$... SiNo Son iguales
- Cuidado con el caso de empate

3.4 ③ Condicional múltiple

Si encadenados o anidados. Cuando hay más de dos caminos posibles.

Clasificar edad (id: clasificar_edad · 10 casos)

Pide la edad de una persona y clasifícala. Como ÚLTIMA línea imprime exactamente UNA de estas palabras:

NIÑO — si $\text{edad} \leq 12$

ADOLESCENTE — si $13 \leq \text{edad} \leq 17$

ADULTO — si $18 \leq \text{edad} \leq 64$

MAYOR — si $\text{edad} \geq 65$

Asume $\text{edad} \geq 0$.

Pistas:

- Encadena varios Si...SiNo o usa Si anidados
- Cuidado con los límites: 12 es NIÑO, 13 es ADOLESCENTE
- Las cuatro palabras van en mayúsculas, sin nada más

Calificación (nota → texto) (id: calificacion · 10 casos)

Pide una nota (0..10) y muestra una categoría:

$\text{nota} < 5 \rightarrow$ INSUFICIENTE

$\text{nota} < 6 \rightarrow$ SUFICIENTE

$\text{nota} < 7 \rightarrow$ BIEN

$\text{nota} < 9 \rightarrow$ NOTABLE

$\text{nota} \leq 10 \rightarrow$ SOBRESALIENTE

Si la nota es < 0 o > 10 , imprime "FUERA DE RANGO".

Como ÚLTIMA línea imprime exactamente una de esas palabras (en mayúsculas).

Pistas:

- Encadena Si/SiNo Si en cascada
- Comprueba primero si está fuera de rango
- Cuidado con los límites: 9 es SOBRESALIENTE, 8.99 es NOTABLE

Mayor de tres números (id: mayor_tres · 10 casos)

Pide tres números y muestra el mayor de los tres.

Como ÚLTIMA línea imprime exactamente:

Mayor: X

Pistas:

- Inicializa una variable mayor con el primer valor
- Compárala con el segundo, después con el tercero
- Usa Si encadenados, no necesitas SiNo

3.5 ④ Bucle Para

Iteración cuando sabes de antemano cuántas veces vas a repetir. Ideal para sumas, tablas, factorial.

Suma de los N primeros pares (id: suma_n_pares · 10 casos)

Pide un número N y calcula la suma de los N primeros números pares positivos:
 $2 + 4 + 6 + \dots + (2 \cdot N)$

Como ÚLTIMA línea imprime exactamente:
Suma: X

Pistas:

- El i-ésimo número par positivo es $2 \cdot i$
- Usa Para $i \leq 1$ Hasta N
- Acumula en una variable suma $\leftarrow$ suma + $2 \cdot i$

Tabla de multiplicar de N (id: tabla_n · 10 casos)

Pide un número N. Imprime su tabla de multiplicar del 1 al 10 con este formato (10 líneas, una por valor):

N x 1 = X

N x 2 = X

...

N x 10 = X

No añadas nada después de la última línea.

Pistas:

- Bucle Para $i \leq 1$ Hasta 10
- Imprime: Escribir n, " x ", i, " = ", $n \cdot i$
- Sin línea final extra

Factorial (id: factorial · 10 casos)

Pide un entero $N \geq 0$ y calcula $N! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot N$. Recuerda: $0! = 1$ (por convenio).

Como ÚLTIMA línea imprime exactamente:
 $N! = X$

(El formato literal: dígito, signo de admiración, espacio, igual, espacio, valor.)

Pistas:

- Inicializa $f \leftarrow 1$
- Bucle Para $i \leq 1$ Hasta n: $f \leftarrow f \cdot i$
- Si $N = 0$, el bucle no se ejecuta y f queda en 1

3.6 ⑤ Bucle Mientras

Iteración cuando NO sabes cuántas veces, sino que dependes de una condición. Patrón centinela.

Sumar hasta introducir 0 (id: sumar_hasta_cero · 10 casos)

Pide números al usuario, uno a uno. Cuando introduzca 0, deja de pedir más y muestra la suma de todos los anteriores (el 0 final NO se suma — es el centinela).

Como ÚLTIMA línea imprime exactamente:
Suma: X

Si el primer número es 0, la suma vale 0.

Pistas:

- Inicializa `suma <- 0` y lee el primer número
- Mientras `n <> 0` Hacer: `suma <- suma + n`, leer el siguiente
- Recuerda imprimir "Suma: " antes del total

Cuenta las cifras de un número (id: cifras_numero · 10 casos)

Pide un entero $N > 0$ y cuenta cuántas cifras tiene.

Como ÚLTIMA línea imprime exactamente:
Cifras: X

Asume $N > 0$ (no se prueba con 0 ni con negativos).

Pistas:

- Inicia `cifras <- 0`
- Mientras `n > 0`: `cifras <- cifras + 1`, `n <- n / 10` (truncado por Entero)

Promedio con centinela -1 (id: promedio_centinela · 10 casos)

Pide números hasta que el usuario introduzca -1 (centinela). Calcula el promedio de los anteriores (el -1 NO se cuenta).

Como ÚLTIMA línea imprime exactamente:
Promedio: X (si hubo al menos un dato)
Sin datos (si el primer número introducido fue -1)

Pistas:

- Cuenta cuántos datos lees
- Acumula `suma <- suma + n`
- `Promedio = suma / contador`
- Antes de imprimir comprueba si `contador > 0`

3.7 ⑥ Algoritmos clásicos

Combinación de bucles y condicionales que resuelven problemas con identidad propia.

Suma de dígitos (id: suma_digitos · 10 casos)

Pide un número entero positivo y calcula la suma de todos sus dígitos.

Por ejemplo:

123 → 1+2+3 = 6

1000 → 1+0+0+0 = 1

999 → 9+9+9 = 27

Como ÚLTIMA línea imprime exactamente:

Suma digitos: X

Pistas:

- Extrae el último dígito con: `digito <- n MOD 10`
- Acumula en suma <- suma + digito
- Reduce n con: `n <- n / 10` (si n es Entero, se trunca automáticamente)
- Repite mientras `n > 0`

¿Es primo? (id: es_primo · 10 casos)

Pide un número entero $N \geq 2$ y determina si es primo (solo divisible entre 1 y sí mismo).

Como ÚLTIMA línea imprime exactamente:

PRIMO o NO PRIMO

Pistas:

- `Bandera es_primo <- Verdadero`
- Recorre divisores con Para $i <= 2$ Hasta $n-1$ (o hasta $\sqrt{n}$)
- Si encuentras un divisor: `es_primo <- Falso`
- Imprime "PRIMO" o "NO PRIMO" según la bandera

Invertir un número (id: invertir_numero · 10 casos)

Pide un entero positivo y muestra el número con sus dígitos invertidos.

Ejemplo: 1234 → 4321. Los ceros a la izquierda del resultado se omiten (10 → 1, no "01").

Como ÚLTIMA línea imprime exactamente:

Invertido: X

Pistas:

- `inv <- 0`
- Mientras `n > 0`: `d <- n MOD 10`, `inv <- inv * 10 + d`, `n <- n / 10`
- Como inv es Entero, los ceros iniciales desaparecen solos

3.8 ⑦ Listas (Dimension)

Trabajar con varios valores en un solo vector. Recorrer, buscar, contar.

Contar pares e impares (id: pares_impares · 10 casos)

Pide N. A continuación pide N números enteros y guárdalos en un array. Cuenta cuántos son pares y cuántos impares.

Recuerda: 0 es par. Los negativos también pueden ser pares o impares (−2 es par, −1 es impar).

Como ÚLTIMA línea imprime exactamente:

Pares: X, Impares: Y

Máximo de un vector (id: max_vector · 10 casos)

Pide N. Después pide N números y guárdalos en un array. Muestra el valor máximo.

Como ÚLTIMA línea imprime exactamente:

Maximo: X (sin tilde para que el grader sea estricto)

Buscar un valor en un vector (id: buscar_valor · 10 casos)

Pide N. Después pide N números. Después pide un valor X a buscar. Devuelve la POSICIÓN (1-indexada) donde aparece la primera ocurrencia de X, o -1 si no aparece.

Como ÚLTIMA línea imprime exactamente:

Posicion: X (sin tilde)

Pistas:

- Usa una bandera o variable pos <- -1
- Recorre con Mientras o Para; para Mientras puedes salir cuando lo encuentres
- Asegúrate de no sobrescribir pos si ya lo encontraste

3.9 ⑧ Menús con Segun

La estructura Segun como selector múltiple, ideal para menús con varias opciones.

Día de la semana (id: día_semana · 10 casos)

Pide un número del 1 al 7 e imprime el día correspondiente. Si está fuera de rango imprime "Día no válido".

1 → Lunes 5 → Viernes
2 → Martes 6 → Sábado
3 → Miércoles 7 → Domingo
4 → Jueves

Como ÚLTIMA línea imprime el nombre del día (con sus tildes correctas) o exactamente "Día no válido".

REQUISITO: usa la estructura Segun (no Si encadenados).

Pistas:

- Usa Segun n Hacer ... FinSegun
- Para el caso fuera de rango usa "De Otro Modo:"
- Cuidado con las tildes: Miércoles, Sábado, Día

Días del mes (id: días_mes · 10 casos)

Pide el número del mes (1..12) y muestra cuántos días tiene. Asume año NO bisiesto (febrero = 28).

Como ÚLTIMA línea imprime exactamente:

Días: X (sin tilde, si mes en rango)

Mes no válido (si fuera de rango)

REQUISITO: usa Segun.

Pistas:

- Meses con 31 días: 1, 3, 5, 7, 8, 10, 12
- Meses con 30 días: 4, 6, 9, 11
- Febrero (2) = 28
- Agrupa con: 1, 3, 5, 7, 8, 10, 12: Escribir "Días: 31"

Nombre del mes (id: nombre_mes · 10 casos)

Pide un número del 1 al 12 y muestra el nombre del mes en español.

Como ÚLTIMA línea imprime exactamente uno de:
Enero, Febrero, Marzo, Abril, Mayo, Junio,
Julio, Agosto, Septiembre, Octubre, Noviembre, Diciembre,
Mes no válido

REQUISITO: usa Segun.

Pistas:

- Cada caso del Segun imprime un nombre
- Cuidado con la ortografía: Septiembre (no Setiembre)

3.10 ⑨ Repetir / Hasta Que

Bucle do-while: se ejecuta al menos una vez y comprueba la condición al final. Ideal para validar entradas.

Adivinar el 7 (id: adivinar_7 · 10 casos)

El número secreto es 7 (fijo, no aleatorio). Pide intentos al usuario hasta que adivine. Cuenta el número de intentos.

REQUISITO: usa Repetir / Hasta Que.

Como ÚLTIMA línea imprime exactamente:
Intentos: X

donde X incluye el intento acertado (si acierta a la primera, X=1).

Pistas:

- Inicializa intentos <- 0 antes del bucle
- Dentro de Repetir: Leer intento, intentos <- intentos + 1
- La condición de salida es: Hasta Que intento = 7

Validar nota 0-10 con Repetir (id: validar_nota_repetir · 10 casos)

Pide una nota hasta que esté entre 0 y 10 (ambos inclusive). Cuando la nota sea válida, imprime como ÚLTIMA línea:
Nota válida: X

REQUISITO: usa Repetir / Hasta Que (no Mientras).

Pistas:

- Repetir { Leer nota } Hasta Que nota >= 0 Y nota <= 10
- Los valores incorrectos NO se imprimen, solo se descartan

Contar hasta múltiplo de 7 (id: contar_hasta_mult7 · 10 casos)

Pide números al usuario uno a uno usando Repetir/Hasta Que. Para cuando introduzca un múltiplo de 7 (incluido el 0, que es múltiplo de 7). Cuenta CUÁNTOS números introdujo en total (incluyendo el múltiplo de 7 final).

Como ÚLTIMA línea imprime exactamente:

Contados: X

Pistas:

- `contados <- 0`
- Dentro de Repetir: `Leer n, contados <- contados + 1`
- Hasta Que `n MOD 7 = 0`

3.11 ⑩ Números aleatorios

Uso de Aleatorio() para simulaciones (dados, monedas, juegos).

Tirar N dados y sumar (id: tirar_dados · 10 casos)

Pide un número N y tira N dados (cada uno con Aleatorio(1, 6)). Suma todos los resultados.

Como ÚLTIMA línea imprime exactamente:

Suma: X

Como X depende del azar, el grader sólo verifica que el formato es correcto y que la suma está en el rango válido (entre N y 6·N, ambos inclusive).

Pistas:

- Usa la función Aleatorio(1, 6) — devuelve un entero de 1 a 6
- Acumula en `suma <- suma + Aleatorio(1, 6)`
- No olvides inicializar `suma <- 0`

■ Como usa Aleatorio, el grader comprueba que (1) el formato sea "Suma: X" y (2) que X esté en el rango [N, 6·N]. No se comprueba el valor exacto.

Lanzar N monedas (id: lanzar_monedas · 10 casos)

Pide un número N. Lanza N monedas con Aleatorio(0, 1): considera 0 = cara, 1 = cruz. Cuenta cuántas caras y cuántas cruces salen.

Como ÚLTIMA línea imprime exactamente:

Caras: X, Cruces: Y

Pistas:

- Usa Aleatorio(0, 1) dentro de un bucle Para
- Si el resultado es 0 sumas caras, si es 1 sumas cruces
- Caras + Cruces siempre tiene que dar N

■ Usa Aleatorio. El grader comprueba que el formato es "Caras: X, Cruces: Y" y que $X + Y = N$. No se comprueban los valores exactos.

Aleatorios > 50 (id: mayores_50 · 10 casos)

Pide N. Genera N números aleatorios entre 1 y 100 con Aleatorio(1, 100). Cuenta cuántos son MAYORES que 50 (estrictamente).

Como ÚLTIMA línea imprime exactamente:
Mayores: X

Pistas:

- Bucle Para i <- 1 Hasta n
- Genera r <- Aleatorio(1, 100)
- Si r > 50 entonces incrementa mayores

■ Usa Aleatorio. El grader comprueba el formato "Mayores: X" y que X está en el rango [0, N].

3.12 ■ Cadenas

Contar consonantes (id: contar_consonantes · 10 casos)

Pide una palabra escrita en minúsculas, sin acentos y sin espacios. Cuenta cuántas consonantes tiene.

Vocales = {a, e, i, o, u}. Cualquier otra letra cuenta como consonante.

Como ÚLTIMA línea imprime exactamente:
Consonantes: X

Pistas:

- Usa n <- Longitud(palabra) para saber cuántos caracteres iterar
- Saca cada letra con: c <- Subcadena(palabra, i, i)
- Comprueba si NO es vocal: c <> "a" Y c <> "e" Y ...

Invertir una palabra (id: invertir_cadena · 10 casos)

Pide una palabra y muéstrala con sus caracteres en orden inverso.

Como ÚLTIMA línea imprime exactamente:
Invertida: X

Pistas:

- Inicializa una cadena vacía: inv <- ""
- Recorre desde i = Longitud(palabra) hasta 1 con paso -1
- En cada iteración: inv <- Concatenar(inv, Subcadena(palabra, i, i))

¿Empieza por vocal? (id: empieza_vocal · 10 casos)

Pide una palabra (escrita en minúsculas, sin acentos). Determina si su primera letra es una vocal (a, e, i, o, u).

Como ÚLTIMA línea imprime exactamente:
SI o NO

Pistas:

- Obtén la primera letra: c <- Subcadena(palabra, 1, 1)
- Comprueba si c = "a" O c = "e" O ...

3.13 ■ Funciones matemáticas

Uso de la constante PI y de funciones RC, Redon, Trunc para fórmulas geométricas.

Área del círculo (id: area_circulo · 10 casos)

Pide el radio R de un círculo y calcula su área:

$$\text{área} = \pi \cdot r^2$$

Muestra el área redondeada al entero más cercano (usa la función Redon).

Como ÚLTIMA línea imprime exactamente:

Area: X

(Usa "Area" sin tilde para que el grader sea estricto.)

Pistas:

- La constante PI ya está definida (no necesitas paréntesis)
- Usa el operador ^ para el cuadrado: r^2
- Redondea con: `a <- Redon(PI * r ^ 2)`

Hipotenusa (Pitágoras) (id: hipotenusa · 10 casos)

Pide los dos catetos A y B de un triángulo rectángulo y calcula la hipotenusa: $h = \sqrt{a^2 + b^2}$.

Muestra el resultado redondeado al entero más cercano (usa Redon y RC).

Como ÚLTIMA línea imprime exactamente:

Hipotenusa: X

Pistas:

- Usa RC para la raíz cuadrada y ^ para el cuadrado
- `h <- Redon(RC(a ^ 2 + b ^ 2))`
- Aprovecha las ternas pitagóricas (3-4-5, 5-12-13, ...) para entender el caso

Distancia entre dos puntos (id: distancia_puntos · 10 casos)

Pide las coordenadas de dos puntos: (x1, y1) y (x2, y2). Calcula la distancia euclidiana:

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

Muestra el resultado redondeado al entero más cercano.

Como ÚLTIMA línea imprime exactamente:

Distancia: X

Pistas:

- Lee los cuatro valores en orden: x1, y1, x2, y2
- `d <- Redon(RC((x2 - x1) ^ 2 + (y2 - y1) ^ 2))`

PARTE IV — Apéndices

4.1 Errores comunes y cómo solucionarlos

Mensaje del intérprete	Qué significa	Solución típica
Variable 'x' usada sin definir	Usaste x antes de declararla con Definir, o escribiste mal el nombre	Añade Definir x Como ... al principio del Proceso, o revisa la ortografía
Se esperaba ASIGN pero se encontró ...	Pusiste = en una asignación en lugar de <-	Cambia x = 5 por x <- 5
Se esperaba FINSI pero se encontró EOF	Te olvidaste de cerrar un Si con FinSi	Cuenta los Si/Mientras/Para abiertos y asegúrate de que cada uno tiene su FinSi/FinMientras/FinPara
Índice x fuera de rango	Accediste a v[i] con i fuera de 1..N donde N es el tamaño del Dimension	Revisa los límites de tus bucles (Para i <- 1 Hasta N, no Hasta N+1)
La condición del Si debe ser lógica	En el Si pusiste un número o cadena en lugar de una comparación	Asegúrate de usar operadores relacionales: Si x > 0 Entonces (no Si x Entonces)
Bucle ... detenido por seguridad tras 1 000 000 iteraciones	Tu Mientras o Repetir nunca termina (la condición no cambia)	Comprueba que actualizas la variable de control dentro del bucle
División por cero	Algún cálculo divide entre 0	Antes de dividir, comprueba con Si que el divisor no es 0
Valor no numérico para 'x': "..."	Declaraste x Como Entero/Real pero el usuario tecleó letras	Asume que el usuario teclea números válidos, o cambia el tipo a Cadena

4.2 Tabla resumen de constructos

Constructo	Sintaxis
Programa	Proceso Nombre ... FinProceso
Definir	Definir x, y Como Entero
Asignar	x <- expresión
Imprimir	Escribir a, b, c
Leer	Leer x Leer v[i]
Si	Si cond Entonces ... SiNo ... FinSi
Segun	Segun expr Hacer N: ... De Otro Modo: ... FinSegun
Mientras	Mientras cond Hacer ... FinMientras
Repetir	Repetir ... Hasta Que cond
Para	Para i <- a Hasta b [Con Paso s] Hacer ... FinPara
Dimension	Dimension v[N] // índices 1..N
Comentario	// el resto de la línea es comentario
Operadores	+ - * / ^ MOD = <> < > <= >= Y O NO

Booleanos	Verdadero Falso
Built-in math	RC Trunc Redon Abs Ln Exp Sen Cos Tan PI
Built-in azar	Azar(N) Aleatorio(a, b)
Built-in texto	Longitud Mayusculas Minusculas Subcadena Concatenar